

Kripto Mesaj - Proje Dokumantasyonu

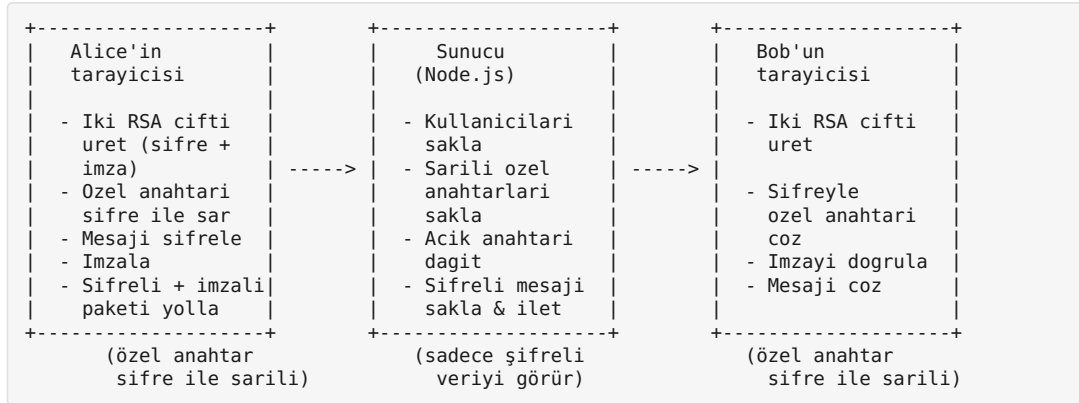
Kripto Mesaj — Proje Dokumantasyonu

1. Projenin amacı

İki kullanıcının birbirine **uctan uca şifreli (end-to-end encrypted, E2EE)** mesaj gönderebildiği bir web sitesi. “Uctan uca şifreli” demek: mesaj göndericinin tarayicisinde şifrelenir, sunucuya yalnızca şifreli hali gider, alıcının tarayicisinde çözülür. **Sunucu mesajın içerigine erişemez** — veritabanında sadece anlamsız bayt dizileri durur.

Bu, WhatsApp/Signal gibi uygulamaların temel çalışma prensibinin sadeleştirilmiş bir versiyonudur.

2. Mimari



2.1 Bileşenler

Katman	Teknoloji	Gorevi
Frontend	HTML + CSS + Vanilla JavaScript	Kullanıcı arayuzu
Şifreleme	Web Crypto API (tarayıcının yerleşik)	RSA-OAEP + RSA-PSS + AES-GCM + PBKDF2
Backend	Node.js + Express	HTTP API
Güvenlik	helmet, express-rate-limit	Güvenlik başlıkları, brute-force koruma
Veritabanı	SQLite (better-sqlite3 , WAL mode)	Kullanıcılar, oturumlar, şifreli mesajlar
Şifre hash	bcryptjs (cost 12)	Kullanıcı şifrelerinin hash'lenmesi
Deploy	Docker / Render / Fly.io	Production hosting

2.2 Klasör yapısı

```

kripto-site/
├── server.js           # Express backend
├── package.json
├── Dockerfile         # Production container imaji
├── .dockerignore
├── render.yaml        # Render.com deploy config
├── fly.toml           # Fly.io deploy config
├── .env.example       # Ornek environment variable'lar
├── public/
│   ├── index.html     # Tek sayfa: giris + chat ekrani
│   ├── styles.css     # Goruntu
│   ├── crypto.js      # Web Crypto API yardimcilari
│   └── app.js         # UI mantigi (auth, mesaj gonder/al)
├── DOKUMANTASYON.md  # Bu dosya
└── README.md

```

3. Sifreleme akisi (en kritik kisim)

Dort farkli kripto primitifini birlikte kullaniriz. Buna **hibrit sifreleme** denir; gercek dunyadaki PGP, S/MIME ve TLS de ayni mantigi kullanir.

Algoritma	Parametre	Kullanim
RSA-OAEP	2048-bit, SHA-256	AES anahtarinin alicinin acik anahtariyla sarilmasi
RSA-PSS	2048-bit, SHA-256	Mesajin gonderici tarafindan imzalanmasi (kimlik dogrulama)
AES-GCM	256-bit + 96-bit IV	Asil mesaj sifrelemesi (gizlilik + butunluk birlikte)
PBKDF2	SHA-256, 310k iter	Kullanici sifresinden ozel anahtarlari saracak anahtar uret
Bcrypt	cost 12	Sunucu tarafinda sifre hash'leme

3.1 Neden bu kadar algoritma var?

- **AES (simetrik):** cok hizli ama her iki tarafin da ayni anahtari bilmesi gerekir. "Ayni anahtari nasil guvenle paylasiriz?" sorusu kalir.
- **RSA-OAEP (asimetrik sifreleme):** yavas ama "acik anahtarla sifrele, ozel anahtarla coz" yapisi sayesinde anahtar paylasimi problemini cozer.
- **RSA-PSS (asimetrik imza):** "ozel anahtarla imzala, acik anahtarla dogrula". Bu olmadan sunucu, kotu niyetli olsa, sahte mesajlar uretip "alice gonderdi" diye gosterebilir. Imza ile alici, gondericinin kimliginden emin olur.
- **PBKDF2:** kullanicinin RSA ozel anahtari sunucuda da bir yerde tutulmali (yoksa baska tarayicidan giremezsin). Ama plaintext olarak tutmak felaket olur. Cozum: kullanicinin sifresinden 310,000 iterasyonla bir AES anahtari turetip ozel anahtari onunla sar. Sifreyi bilmeyen (sunucu dahil) acamaz.
- **Bcrypt:** sifre hashing'i icin standart secim. 12 cost factor su an icin yeterli yavaslikta (~250ms/hash).

3.2 Kayit akisi (adim adim)

public/app.js icinde register-form event handler:

1. Tarayici **iki RSA cifti uretir**: biri RSA-OAEP (sifreleme), biri RSA-PSS (imza).
2. **Rastgele 16 bayt tuz (salt)** uretilir.
3. Kullanici sifresinden PBKDF2 ile **AES-GCM-256 sarma anahtari** turetilir (310,000 SHA-256 iterasyonu).
4. Her iki ozel anahtar bu sarma anahtariyla AES-GCM ile sarilir.
5. Sunucuya yollanir:

- `username`, `password` (sunucu bcrypt'leyip saklar)
 - `encPublicKey`, `signPublicKey` (acik halde)
 - `wrappedEncPrivateKey`, `wrappedSignPrivateKey`, `keySalt`, `encPrivIv`, `signPrivIv` (ozel anahtarlar sifreyle sarilmis halde — sunucu acamaz)
6. Sunucu kayıt yapar ve oturum token'i doner.
 7. Tarayici, ham ozel anahtarlar **sessionStorage**'da tutar (tab kapaninca silinir).

3.3 Giris akisi (adim adim)

1. Kullanici `username` + `password` gonderir.
2. Sunucu bcrypt ile sifreyi dogrular ve oturum token'i + sarili anahtarlar doner.
3. Tarayici, kullanicinin sifresinden ayni PBKDF2 ile sarma anahtarini yeniden uretir.
4. Sarili ozel anahtarlar acar (AES-GCM-decrypt).
5. Anahtarlar sessionStorage'a yazilir.

Yanlis sifre verilirse: AES-GCM cozme **butunluk dogrulaması** sayesinde basarisiz olur ve hata firlatir (AES-GCM bir AEAD mod, sifre yanlissa veri reddedilir — magic).

3.4 Mesaj gonderme akisi

`KripoCrypto.encryptAndSign` (public/crypto.js):

1. **Rastgele AES-GCM-256 anahtari uret.** (Her mesaj icin yeni — bir mesaj ele gecirilirse digerleri etkilenmez.)
2. **Rastgele 12 bayt IV uret.**
3. **Mesaji UTF-8'e cevir** ve AES-GCM ile sifrele → `ciphertext`.
4. AES anahtarini ham bayt olarak dis ari aktar (32 bayt).
5. Alicinin RSA-OAEP acik anahtarini sunucudan al.
6. **AES anahtarini alicinin RSA-OAEP acik anahtariyla sifrele** → `encryptedKey`.
7. **Imzalama yuk metni hazirla:** `sender|recipient|signedAt|iv|encryptedKey|ciphertext` (Buradaki tum alanlar imzaya dahil; saldirgan herhangi birini degistirirse imza dogrulaması basarisiz olur.)
8. Bu yuku **gondericinin RSA-PSS ozel anahtariyla imzala** → `signature`.
9. Sunucuya gonder: `{ to, ciphertext, iv, encryptedKey, signature, signedAt }`.

Sunucu ek olarak: - `signedAt` 'in 5 dakikadan eski/ileri olmadigini kontrol eder (replay koruma). - `to != sender` zorunlu. - Rate limit: dakikada 60 mesaj.

3.5 Mesaj alma akisi

`KripoCrypto.verifyAndDecrypt`:

1. Gondericinin RSA-PSS acik anahtarini sunucudan al (cache'le).
2. Ayni imzalama yuk metnini yeniden kur.
3. **Imzayi dogrula.** Basarisizsa mesaj UI'da **kirmizi cerceveyle "IMZA GECERSIZ"** etiketiyle gosterilir.
4. `encryptedKey` 'i kendi RSA-OAEP ozel anahtariyla coz → AES anahtarini elde et.
5. AES anahtarini Web Crypto'ya import et.
6. `ciphertext` 'i AES-GCM ile coz (IV ile birlikte) → UTF-8 plaintext.

3.6 Anahtar yonetimi

- **İki çift RSA anahtari** vardır: sifreleme (RSA-OAEP) ve imza (RSA-PSS). Cryptographic best

practice: bir anahtarın sadece bir görevi olsun.

- **Acik anahtarlar** sunucuda düz halde durur, herkese dağıtılır.
- **Ozel anahtarlar** sunucuda **sifre ile sarili** halde durur. Sunucu açamaz.
- Çalışma sırasında açılmış özel anahtarlar tarayıcının **sessionStorage**'inde durur — tab kapatıldığında otomatik silinir.
- Sayfa yenilenirse (tab açıkken) anahtarlar hala sessionStorage'da; refresh'te sorun olmaz. Tab kapatılırsa tekrar şifre ile kilidi açılır (unlock ekranı).

4. Güvenlik önlemleri

Konu	Çözüm
Sunucudan sızıntı	Ozel anahtarlar AES-GCM ile sarili; PBKDF2 (310k iter) lazim
Sunucu sahte mesaj iddiası	Her mesaj RSA-PSS ile imzalı, alıcı doğruluyor
Sifre brute-force	bcrypt cost 12 + login için 15 dk/20 deneme rate-limit
Mesaj flood	Dakikada 60 mesaj rate-limit
Replay (eski mesajın tekrarlı)	signedAt zaman damgası imzaya dahil + 5 dk pencere
Mesaj kurcalama	RSA-PSS imzası tüm alanları korur + AES-GCM bütünlük tag'i
XSS	Helmet CSP (sadece self, inline yok);textContent kullanılıyor
Clickjacking	frame-ancestors: 'none' + X-Frame-Options DENY
MITM	HSTS + HTTPS yönlendirme (production'da)
Oturum bekleme	Token 7 gün sonra otomatik süresi dolar, saatlik temizlik
Username enumeration	Login'de sabit-zamanlı bcrypt karşılaştırma (phony hash)
SQL injection	Tüm sorgular prepared statement
Path traversal	Express.static + sanitized parametre
Bilgi sızıntısı (stack trace)	Production hata yakalayıcısı sadece {error} döner

4.1 Hala kalan eksiklikler (rapor için doğruluk)

- **Forward secrecy yok.** Aynı RSA çifti tüm mesajlarda kullanılır. Anahtar çifti ele geçirilirse tüm geçmiş mesajlar çözülebilir. Tam çözüm Signal'in Double Ratchet protokolu — okul projesi için fazla karmaşık.
- **Meta-veri sunucuda görülüyor.** Kim-kime-ne-zaman bilgisi şifreli değil.
- **Sifre kurtarma yok.** Kullanıcı şifresini unutursa eski mesajlarını sonsuza kadar kaybeder. Bu E2EE'nin doğal sonucu (Signal'da da böyledir).

5. Backend API

Tüm istekler JSON, kimlik doğrulama Authorization: Bearer <token> header'i ile.

Metod	Yol	Aciklama
GET	/api/health	Servis ayakta mi (deploy health-check icin)
POST	/api/register	Yeni kullanıcı + sarili anahtarlar
POST	/api/login	Sifre ile giris, sarili anahtarlarini geri doner
POST	/api/logout	Oturumu silmek
GET	/api/me	Mevcut kullanicinin sarili anahtarlarini al
GET	/api/users	Diger kullanicilarin listesi
GET	/api/users/:username/keys	Bir kullanicinin iki acik anahtari (enc+sign)
POST	/api/messages	Sifreli + imzali mesaj gonder
GET	/api/messages?with=X&since=T	Belirli bir kullanıcıyla mesajlar

5.1 Veritabani semasi

```

users (
  username          TEXT PRIMARY KEY,
  password_hash     TEXT NOT NULL,      -- bcrypt cost 12
  enc_public_key    TEXT NOT NULL,      -- base64 SPKI (RSA-OAEP)
  sign_public_key   TEXT NOT NULL,      -- base64 SPKI (RSA-PSS)
  wrapped_enc_private_key TEXT NOT NULL, -- AES-GCM ile sarili
  wrapped_sign_private_key TEXT NOT NULL, -- AES-GCM ile sarili
  key_salt          TEXT NOT NULL,      -- PBKDF2 tuzu
  enc_priv_iv       TEXT NOT NULL,      -- AES-GCM IV (enc icin)
  sign_priv_iv      TEXT NOT NULL,      -- AES-GCM IV (sign icin)
  created_at        INTEGER
)

sessions (
  token            TEXT PRIMARY KEY,
  username         TEXT NOT NULL,
  created_at       INTEGER,
  expires_at       INTEGER              -- Oturum suresi (7 gun varsayilan)
)

messages (
  id               INTEGER PRIMARY KEY,
  sender           TEXT NOT NULL,
  recipient        TEXT NOT NULL,
  ciphertext       TEXT NOT NULL,      -- base64, sunucu ICERIGI GORMUYOR
  iv              TEXT NOT NULL,
  encrypted_key    TEXT NOT NULL,      -- base64 RSA-OAEP ciklisi
  signature        TEXT NOT NULL,      -- base64 RSA-PSS imzasi
  signed_at        INTEGER NOT NULL,    -- replay korumasi icin
  created_at       INTEGER
)

```

6. Demo nasıl yapılır

6.1 Lokal demo (tek bilgisayar)

```

npm install
npm start

```

Sonra: 1. `http://localhost:3000` adresini ac. 2. Bir pencerede **Kayıt ol** → `alice / gizli12345`. 3. Aynı siteyi **gizli pencerede** ac → **Kayıt ol** → `bob / gizli67890`. 4. Alice'in penceresinde sol listede `bob`'u sec, mesaj yaz. 5. Bob'un penceresinde sol listede `alice`'i sec — mesaj cozulmus olarak gelir. 6. **“Sifreli icerigi goster”** kutusunu tikla → her mesajin altinda sunucuya giden ciphertext, IV, sifrelenmis AES anahtari ve imza goruntulenir. 7. Her mesajda **“✓ imza dogru”**

etiketi görünmeli.

6.2 Sunucuda hiçbir şeyin görünmediğini doğrulama (hocaya demo)

Server çalışırken, ayrı bir terminalde:

```
sqlite3 kripto.db "SELECT sender, recipient, substr(ciphertext, 1, 40) FROM messages;"
```

Cikti soyle gozukur:

```
alice|bob|P0UtgcGEIz3fdhkm6zomF36e75mZ1C9PXZZy2rL...
```

`ciphertext` sütununda hiçbir okunabilir metin yok. Mesaji okuyabilmek için alıcının **sifre ile acılmış** özel anahtarına ihtiyac var; o da yalnızca alıcının tarayıcısında ve sifre bilgisiyle erişilebilir.

Ekstra demo: `users` tablosuna bak:

```
sqlite3 kripto.db "SELECT username, substr(wrapped_enc_private_key, 1, 40) FROM users;"
```

Özel anahtarların sarılı (anlamsız bayt) durduğunu göster.

7. Production deployment

7.1 Render.com (kullanılan host)

Bu projenin canlı sürümü Render.com ücretsiz tier'da yayınlandı.

- **Canlı URL:** <https://kripto-site.onrender.com/>
- **Region:** Frankfurt (EU Central)
- **Runtime:** Docker (Dockerfile multi-stage build)
- **Otomatik deploy:** `main` 'e push edildiğinde Render kendiliğinden yeniden build alır.

Sifirdan kurmak için: GitHub ile Render hesabı aç → **New + Web Service** → `reverx99/kripto-site` repo'sunu seç → Branch `main`, Language Docker, Instance Type Free, `NODE_ENV=production` → Deploy. Detaylı adım adım talimat için proje köklerindeki `DEPLOY.md` dosyasına bak.

⚠ Render free tier'da kalıcı disk yok. 15 dakika kullanılmazsa servis uyur, uyandığında SQLite dosyası sıfırlanır. Demo sırasında yeni hesaplar açıp canlı mesajlaşarak göstermek bu kısıtlamayı avantaja çevirir — hocaya sunucuda hiçbir on-veri olmadığı sıfır noktasından başlandığı gösterilir.

7.2 Fly.io (kalıcı veri, biraz daha karmaşık)

```
# flyctl kur: https://fly.io/docs/hands-on/install-flyctl/
flyctl auth signup
flyctl launch --copy-config --no-deploy # app ismini onayla
flyctl volumes create data --size 1 --region fra
flyctl deploy
```

Fly.io free tier'da 3GB volume hakkın var, SQLite kalıcı olarak durur.

7.3 Docker (kendi VPS'inde)

```
docker build -t kripto-site .
docker run -d \
  -p 80:3000 \
  -e NODE_ENV=production \
  -v $(pwd)/data:/data \
  --name kripto kripto-site
```

HTTPS için onune **Caddy** veya **nginx + Let's Encrypt** koy.

7.4 Yedek strateji

Bu proje için tek bir canlı URL (Render) yeterli görüldü. Demo günü için yedek planı:

1. **Demo'dan ~3 dakika önce** Render URL'ini aç → cold start uyumussa ~30 saniye bekle, ana sayfa açılana kadar emin ol.
2. **Lokal yedek:** Sunum yapacağın laptop'a repo'yu klonla, `npm install` ve `npm start` ile lokalde çalışır halde tut. Render demo sırasında olur da tamamen düşerse `http://localhost:3000` ile devam edilebilir.
3. **GitHub repo** zaten kalıcı bir yedek (kaynak kod kaybolmaz).

Not: Glitch.com 2025'te kapatıldı, "mirror deploy" için önceden tavsiye edilen alternatif artık mevcut değil. Free tier'da Node.js destekleyen ve kredi kartı istemeyen seçenekler azaldı; tek host + lokal yedek bu seviye proje için yeterli.

8. Kod akisinin özeti

8.1 Kayıt

```
[Tarayıcı]
KC.generateEncryptionKeyPair()           // RSA-OAEP-2048
KC.generateSigningKeyPair()              // RSA-PSS-2048
salt = random(16 bytes)
wrappingKey = PBKDF2(password, salt, 310k iter)
wrappedEncPriv = AES-GCM.encrypt(encPriv_pkcs8, wrappingKey)
wrappedSignPriv = AES-GCM.encrypt(signPriv_pkcs8, wrappingKey)
POST /api/register { username, password, encPub, signPub,
                      wrappedEncPriv, wrappedSignPriv, salt, iv'ler }

[Sunucu]
bcrypt.hash(password, cost=12)
INSERT INTO users
yeni rastgele 32-byte session token
INSERT INTO sessions
→ { token, expiresAt }
```

8.2 Mesaj gönderme

```
[Alice'in tarayicisi]
GET /api/users/bob/keys // { encPublicKey, signPublicKey }
KC.encryptAndSign(text, bobEncPub, aliceSignPriv, 'alice', 'bob')
  aes = AES-256-GCM rastgele anahtar
  iv = 12 bayt rastgele
  ciphertext = AES-GCM.encrypt(text, aes, iv)
  encryptedKey = RSA-OAEP.encrypt(aes, bobEncPub)
  signedAt = Date.now()
  payload = sender|recipient|signedAt|iv|encryptedKey|ciphertext
  signature = RSA-PSS.sign(payload, aliceSignPriv)
POST /api/messages { to:'bob', ciphertext, iv, encryptedKey, signature, signedAt }

[Sunucu]
signedAt'i ±5dk kontrol et
to != sender kontrol et
rate-limit kontrol et (60/dk)
INSERT INTO messages // sadece sifreli + imza saklanir
```

8.3 Mesaj alma

```
[Bob'un tarayicisi]
Her 2 saniyede:
GET /api/messages?with=alice&since=<son zaman>
Her gelen mesaj icin:
  GET /api/users/alice/keys (cache'li)
  payload = sender|recipient|signedAt|iv|encryptedKey|ciphertext
  verified = RSA-PSS.verify(signature, payload, aliceSignPub)
  aes = RSA-OAEP.decrypt(encryptedKey, bobEncPriv)
  text = AES-GCM.decrypt(ciphertext, aes, iv)
  Ekranaya yaz + imza durumunu goster
```

9. Olasi gelistirmeler (zaman kalirsa)

- WebSocket ile gercek zamanli iletim (polling yerine)
- Dosya/resim gonderme (ayni hibrit yontemle sifrelenir)
- Anahtar parmak izi (fingerprint) gosterme — arkadasinla telefonla teyit et
- Forward secrecy (Double Ratchet)
- Kendini imha eden mesajlar / mesaj silme
- WebAuthn / passkey ile sifresiz giris
- Grup sohbeti

10. Kaynaklar

- MDN — Web Crypto API: https://developer.mozilla.org/en-US/docs/Web/API/Web_Crypto_API
- NIST FIPS 197 (AES): <https://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.197.pdf>
- RFC 8017 (PKCS #1 / RSA): <https://www.rfc-editor.org/rfc/rfc8017>
- OWASP Password Storage Cheat Sheet (PBKDF2 iter sayisi): https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html
- Signal protocol genel bakis: <https://signal.org/docs/>
- Helmet (security headers): <https://helmetjs.github.io/>